arXiv:2609.30462v1 [cs.RO] 24 Sep 2026

Policy-Calibrated DAgger: Offline Calibrated Noise Injection for Imitation Learning

Jenny Wang[1] and George Kantor[1]

***Abstract*— Policies trained with imitation learning can accumulate errors over time, causing the robot to drift outside the training distribution. Existing methods mitigate this covariate shift by collecting additional data where the policy fails or is likely to fail. The first places the robot in unsafe conditions and the second requires choosing an appropriate noise distribution to collect new expert demonstrations under that noise. We propose Policy-Calibrated DAgger, a method that makes use of the properties of recent generative policies to estimate the policy's noise offline by using its own predicted action distribution. We measure a diffusion policy's spread of predicted actions at observations along the expert trajectory and measure its closed-loop error relative to a recorded trajectory. To address issues with measuring error in a multimodal action space, we guide the policy towards the trajectory during closed-loop control through partial denoising, and use properties of a diffusion model to unnormalize the measured error as if we did not guide it. We experiment in a scenario where a robot is tasked to reach an engine lever in a cluttered and narrow environment and show results in a 3D photorealistic simulator and a 2D planar reacher environment. We show that our method surpasses policies trained with dataset aggregation without noising and matches the performance of the best noise level in hindsight, without requiring a sweep over noise levels.**

I. Introduction

Behavior cloning is a popular method that enables scalable learning for robot manipulation [1], [2], [3], [4]. However, its performance is limited by compounding errors over time, as small policy errors cause the state distribution to drift away from the training data [5], [6]. DAgger mitigates this covariate shift by aggregating expert data within the policy's induced state distribution, yielding a policy whose errors scale linearly rather than quadratically with time [5]. DAgger relies on alternating control between the expert and the policy in order for the expert to provide guidance within the policy's induced state distribution. However, this makes the system unintuitive for a human expert to control.

In practice, many methods do not set the frequency at which the expert is queried, but instead let the expert return control to the policy at their convenience [7], [8], [9]. This enables higher quality expert data at the cost of covering states less aligned with the policy's induced state distribution. Recent success has been found by equipping human experts with teleoperation devices that track the real robot with force feedback, enabling human experts to give corrective actions closer to the state distribution induced by the policy [10], [11]. However, these methods place the burden of distribution matching on the expert, which is labor-intensive.

[1]J. Wang and G. Kantor are with Carnegie Mellon University, Pittsburgh PA, USA {jennyw2, gkantor}@andrew.cmu.edu



Fig. 1: Our method estimates policy error while tracking a trajectory by making use of the probabilistic properties of generative policies. By sampling along pre-recorded expert data and making use of the model's noise schedule to undo the normalizing effects of partial denoising towards the expert distribution in closed-loop control in simulation, we can estimate the policy's noise with respect to the expert data. We can sample targeted recovery trajectories within this policy-calibrated noise distribution to reduce covariate shift between the policy's visited states and the expert data.

Our key insight is that closing the gap between the state and action distributions induced by the expert and the policy is essential for addressing covariate shift in practice. This is important for long-horizon tasks requiring precise, full-body maneuvers like a robot arm reaching around a narrow and cluttered engine room. We propose a method that uses the policy to adapt expert intervention data to the policy's learned action distribution, without requiring the expert to control a noisy system or modifying policy training. We build on partial diffusion [12], which adds noise to a sample of the output distribution and uses the model to denoise it to a similar result. We use it to simulate policy-noise-expert interactions probabilistically, without requiring the expert to adapt to a noisy system in the loop. Our policy-calibrated noise, when paired with Gaussian noise and action relabeling to connect augmented states back to the expert intervention, trains policies with better recovery behaviors by limiting covariate shift.

We draw close comparisons to data augmentation techniques for imitation learning that expand a limited set of expert demonstrations to cover the robot's workspace. Covering all possible states requires compute and modeling capacity. In addition, many approaches require an oracle

policy or additional inductive biases about the environment. We show that our method expands the diversity of limited expert demonstrations in a way that targets task failures without needing an oracle policy or additional real-world data collection.

In this work, we propose a method to estimate policy-calibrated noise that reduces covariate shift between the state distributions induced by the policy and the expert. Our core contributions are as follows:

- We propose policy-calibrated data augmentations, a data augmentation method that noises expert interventions to reduce covariate shift with respect to the policy's induced state distribution, generating highly relevant recovery state-action pairs.
- We extend partial denoising [12] to action chunking.
- We show that our method increases data efficiency and policy performance in a long-horizon, cluttered goal-reaching task. Our experiments are performed on a photorealistic simulator of our real engine lever-reaching task mockup and also in a 2D planar simulator that encapsulates the narrow passageways in the real task.

II. Related Work

A. Interactive Imitation Learning

DAgger [5] methods address the data mismatch and compounding error issues in behavior cloning. In theory, DAgger at convergence will collect expert recovery actions in parts of the state space that the policy would visit. DART [6] shows that aggregating expert demonstrations within a certain noise level of the optimal policy reduces covariate shift. This theoretically mitigates the data mismatch problem. In practice, the constant switching between policy actions and expert actions makes it difficult for experts to provide corrective input due to system oscillations [13]. Some methods tackle this data-intensive process with end-to-end automated world models [14], [15], only querying the expert when the model is uncertain [16], [17], [18], [19], or when the state has high variance [20]. HG-DAgger [7] is a variant of DAgger that lets the expert decide whether to execute policy actions or their own actions in the system. The expert is instructed to give corrective inputs whenever the system enters an unsafe state, then return control to the policy once the system is safe again. Dataset aggregation methods designed with human-gated intervention tend to show strong empirical results [8], [10], [11] despite long stretches of expert actions that may move the system far from the policy distribution. Recent work has focused on residual interventions and providing the human expert with force feedback to collect error intervention data without interrupting the current robot policy [21], [8]. This allows the expert interventions to stay close to the policy's induced state distribution by design. However, this places the burden of matching the policy on the expert. We propose a method that makes the policy adapt to the expert.

B. Learning from Suboptimal Data

Some imitation learning methods treat suboptimal data as a noisy ground truth signal. For example, pretraining a model on an auxiliary task often enables data-efficient training on the target task if the two tasks are aligned [22], [23], [24]. Also, simulated data can supplement limited real-world training datasets to increase task success rate when co-training [25]. Iteratively fine-tuning a policy on a continuously changing robot embodiment can increase the efficacy of policy transfer from a data source with a large embodiment gap [26], [27].

Other methods use noisy data to shape what expert intervention data should be collected. This can be done by querying the expert when a policy is uncertain [16], [17], [18], [19] or by injecting noise into the expert data distribution [6], [28], [29]. We observe that the policy is a useful model of its own errors. We use partial diffusion to sample the noise model of the policy. Partial diffusion has previously been studied in shared autonomy [12], [30] and image editing [31]. Its result is bounded by some distance to the original action [31]. We show that this bound lets us estimate the policy's error relative to the expert while constraining the policy to move similarly to the expert during trajectory sampling in offline simulations. In addition, we show that the targeted data diversity obtained by noise calibration can be used to improve the policy's robustness with local action augmentations without additional data collection.

C. Data Augmentation for Imitation Learning

Collecting demonstrations for imitation learning is a labor-intensive process. Many works propose methods to expand expensive expert data into a wider range of data through data augmentations. Learned semantic localization [32] and LLM priors [33] can inform object and object configuration randomization in an environment. Certain object classes can be generated heuristically at scale [34]. In addition, for a known scenario, an oracle planner in a simulated environment can generate new trajectories within demonstrator-annotated [35] or task-specific [29], [28] precision from the original, generate new paths around detected 3D obstacles [36], and retarget demonstrations to perturbed object configurations either in a simulator [37], [38] or in an edited 3D point cloud [39]. We propose a method to expand expert demonstrations to a more diverse set of trajectories without requiring an oracle policy or additional inductive biases. Our proposed data augmentation method generates recovery trajectories targeting areas of the state space that the policy would likely visit due to errors in following the expert data. We show that this targeted augmentation generated after calibrating with the policy's error performs as well as using the result of a noise level hyperparameter sweep.

III. Preliminaries

A. Noise Injection for Robust Imitation Learning

We aim to minimize covariate shift by optimizing the KL-divergence between the trajectories visited by the trained robot policy $p(\xi|\pi_\theta)$ and those visited by the noised expert $p(\xi|\pi_{\theta^*}, \psi)$.

In DART [6], the authors minimize covariate shift by optimizing the KL-divergence with respect to the sufficient

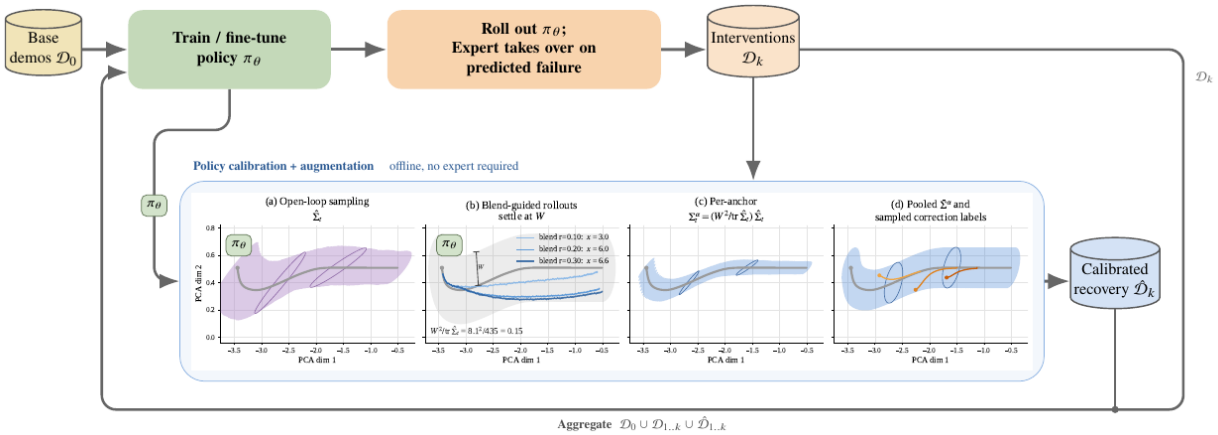


Fig. 2: Our method augments the data collected during a DAgger-style data collection loop. The policy at data collection round k is trained on the base dataset $\mathcal{D}_0$, aggregated intervention data $\mathcal{D}_1 \ldots \mathcal{D}_k$ and noised intervention data $\hat{\mathcal{D}}_1 \ldots \hat{\mathcal{D}}_k$. To generate policy-calibrated noised intervention data: **(a)** We sample the policy open loop at every other timestep along the expert trajectory to estimate $\hat{\Sigma}_t$. **(b)** We sample blended trajectories between the policy and expert with partial denoising to estimate W. **(c)** We combine (a) and (b) to compute the policy noise per timestep Σ_t^α. **(d)** We average the per-timestep Σ_t^α across all data to obtain the pooled $\bar{\Sigma}^\alpha$. In this example, the per-timestep ellipsoidal noise estimates average to become more isotropic. We sample states and corrective action pairs within this local area around the expert intervention.

statistics ψ of the noise distribution. They derive an iterative algorithm to compute Σ_{k+1}^α to estimate the noise parameter of a Gaussian with covariance Σ:

$$\hat{\Sigma}_{k+1} = \frac{1}{T} E_{p\left(\xi \mid \pi_{\theta^*}, \Sigma_k^\alpha\right)} \sum_{t=0}^{T-1} \left(\pi_\theta\left(s_t\right) - \pi_{\theta^*}\left(s_t\right)\right)\left(\pi_\theta\left(s_t\right) - \pi_{\theta^*}\left(s_t\right)\right)^T \tag{1}$$

$$\Sigma_{k+1}^\alpha = \frac{\alpha}{T \operatorname{tr}\left(\hat{\Sigma}_{k+1}\right)} \hat{\Sigma}_{k+1} \tag{2}$$

This requires querying the expert at noised states $\xi \sim p\left(\xi \mid \pi_{\theta^*}, \Sigma_k^\alpha\right)$ in closed loop, which is labor-intensive. In addition, the estimate of the robot's final error α requires hyperparameter tuning, so the process must be repeated.

IV. Policy-Calibrated DAgger

We introduce Policy-Calibrated DAgger, a method that predicts data augmentation noise parameters that make the expert demonstrations cover states closer to the final robot's distribution. A key difference between our method and DART [6] is that we do not inject noise into the expert's control loop—instead, we compute the sufficient statistics offline and in a simulator. This places the burden of corrections on the policy instead of the expert and allows the expert to provide clean demonstrations, uninterrupted.

A. Method Design

We present a method that uses policy-calibrated data augmentations in the DAgger training loop. We generate targeted recovery trajectories that match policy failures during evaluation using a noise metric computed entirely offline by exploiting known noise schedules in generative models. Our system is designed as follows, as shown in Figure 2:

1) Train a policy with imitation learning using some base dataset of expert demonstrations $\mathcal{D}_0$.
2) Aggregate expert intervention data from the kth round of DAgger $\mathcal{D}_k$ into the training dataset to create the aggregate dataset $\mathcal{D}_{agg,k} = \mathcal{D}_0 \cup \mathcal{D}_1 \cup \ldots \cup \mathcal{D}_k$.
3) Generate augmented intervention data $\hat{\mathcal{D}}_k$ from the intervention data $\mathcal{D}_k$ by noising states from the recorded expert intervention data $\mathcal{D}_k$ and generating a trajectory that returns to the expert trajectory. See subsection IV-D for details. This creates the aggregate augmented dataset $\hat{\mathcal{D}}_{agg,k} = \hat{\mathcal{D}}_1 \cup \hat{\mathcal{D}}_2 \cup \ldots \cup \hat{\mathcal{D}}_k$ (created on the fly in the training loop in practice).
4) Co-train the policy on the aggregate dataset $\mathcal{D}_{agg,k}$ and the augmented intervention data $\hat{\mathcal{D}}_{agg,k}$.
5) Repeat steps 2-4 for the number of dataset aggregation rounds.
6) For final evaluation statistics, we fine-tune Step 1's policy π_0 on all rounds' aggregate and noised datasets.

To collect intervention data, we follow HG-DAgger [7] and collect expert-gated interventions. This is in contrast to DAgger, which switches between the policy and the expert at each timestep and may make it hard for the expert to provide demonstrations. For reproducibility in our experiments, we do not use human expert data but instead use a deterministic RRT-based oracle. The oracle uses privileged information in a simulator to decide when to return control back to the policy based on heuristics like whether the policy's predicted action chunk will drive the robot into collision or if the policy has not yet reached the goal after a given number of timesteps.

B. Partial Denoising as a Learned Action Interpolator

We leverage a generative policy's properties to model its deviation from the expert data. Because the policy may solve the task in a different way from the expert, making common

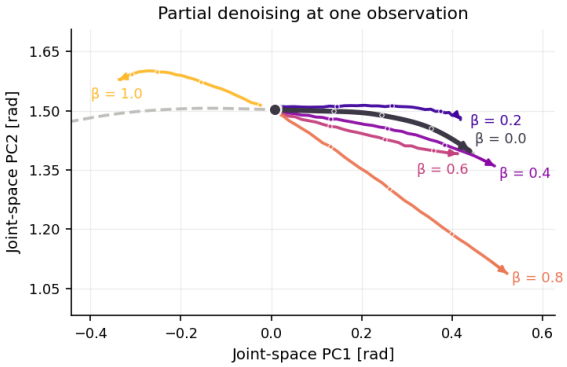


Fig. 3: Empirically, performing partial denoising at lower blend ratios β results in action chunks closer to the sampled expert chunk. The unguided policy $\beta = 1.0$ may sometimes travel in a different direction than the sampled expert chunk.

error metrics ill-posed, we interpolate between the expert and policy actions with partial denoising. Then, we use the policy's noise schedule to recover an estimate of the policy's true error with respect to the expert data.

At state s_t, the robot policy π_θ sees an observation $o_t \in \mathbb{R}^n$ and predicts action $a_{\theta,t} \in \mathbb{R}^d$ at timestep t. The expert prefers the action $a_{\theta^*,t} \in \mathbb{R}^d$ at state s_t. A common method for blending actions between an expert and a policy is linear interpolation [40], [41]. With linear interpolation, the blend ratio $\beta \in [0, 1]$ controls the extent to which the expert's action is preserved relative to the robot action in the blended action $\hat{a}_{t,\text{lerp}}$, where $\beta = 0$ is fully the expert action and $\beta = 1$ is fully the policy action:

$$\hat{a}_{t,\text{lerp}} = \beta a_{\theta,t} + (1 - \beta) a_{\theta^*,t} \tag{3}$$

Linear interpolation can fail to produce a valid action even if both $a_{\theta,t}$ and $a_{\theta^*,t}$ are valid. For example, imagine a scenario where the robot must avoid an obstacle ahead of it. Also, assume that $a_{\theta,t}$ represents avoiding the obstacle by going to the left, and $a_{\theta^*,t}$ represents avoiding the obstacle by going to the right. The blended action $\hat{a}_t$ at $\beta = 0.5$ would crash the robot into the obstacle by going forwards. A learned interpolator may project the action space in a way that the middle of the two actions is semantically meaningful.

We can leverage learned priors in the policy π_θ to blend $a_{\theta^*,t}$ and $a_{\theta,t}$ in a semantically meaningful way:

$$\hat{a}_{t,\theta} = f_\theta(a_{\theta^*,t}, \beta | o_t) \tag{4}$$

Here, the policy parameters θ implicitly encode the robot action $a_{\theta,t}$.

One method of blending with learned priors is partial denoising [12], [31]. Here, we will describe partial denoising for a diffusion model. A diffusion model is a generative model with a forward diffusion process and a reverse diffusion process [42]. The forward diffusion process adds noise to a sample from a data distribution and the reverse diffusion process denoises a noisy input. One step in the forward diffusion process can be written as:

$$a_{\theta^*,t,k} = \sqrt{\alpha_k} a_{\theta^*,t,k-1} + \sqrt{1 - \alpha_k}\epsilon, \epsilon \sim \mathcal{N}(0, I) \tag{5}$$

for a noise schedule of K steps $\beta_1, \ldots, \beta_K$ and $\alpha_k := 1 - \beta_k$.

The reverse diffusion process is derived as:

$$a_{\theta^*,t,k-1} = \frac{1}{\sqrt{\alpha_k}}(a_{\theta^*,t,k} - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}}\epsilon_\theta(a_{\theta^*,t,k}, k | o_t)) + \sigma_k z \tag{6}$$

where $\bar{\alpha}_k := \prod_{s=1}^{k} \alpha_s$, $z \sim \mathcal{N}(0, I)$, and β_k are the diffusion noise schedule coefficients.

Partial diffusion is performed as follows for a forward diffusion ratio $\beta \in [0, 1]$: 1) The switching timestep is $k_{sw} = \text{ToInteger}(\beta K)$, 2) We run the forward diffusion process for $k_{sw} - 1$ timesteps with the expert action $a_{\theta^*,t}$ as input, 3) We run the reverse diffusion process for the same $k_{sw} - 1$ timesteps on the result of the second step. More details can be found in [12].

The extent to which the original action is preserved is bounded by the number of forward and reverse diffusion process steps performed. More formally, as derived in [31], assuming for the predicted noise $\epsilon_\theta(a, k|o)$ that $||\epsilon_\theta(a, k|o)|| \leq \mathcal{K}\ \forall a \in \mathcal{A}$ for $k \in [0, K]$ and actions $a \in [-1, 1]$ are normalized, then for all $\delta \in (0, 1)$ with probability at least $(1 - \delta)$:

$$||a_{\theta^*,t} - \hat{a}_{t,\theta}||_2^2 \leq \sigma_{k_{sw}}^2(\mathcal{K}\sigma_{k_{sw}}^2 + d + 2\sqrt{-d \cdot \log\delta} - 2\log\delta) \tag{7}$$

where d is the dimensionality of a. Empirically, it has been shown that a larger number of diffusion steps k_{sw} results in larger differences between the original action $a_{\theta^*,t}$ and the action resulting from partial diffusion $\hat{a}_{t,\theta}$ [12]. See Figure 3.

C. Extending Partial Denoising to Action Chunks

We extend partial denoising to action chunks $\overrightarrow{\boldsymbol{a}}_{\theta^*,t:t+H} \in \mathbb{R}^{d \times H}$ of time horizon H. Theoretically, the method remains the same because we can flatten the $\overrightarrow{\boldsymbol{a}}_{\theta^*,t:t+H}$ matrix into a vector. In practice, this means we must request actions from the expert that span some time into the future. We circumvent this by processing expert data offline and leave the problem of expert intent inference to future work. The use of action chunks means that an expert must intervene for at least H time steps for the policy to be able to learn from the data. We discard interventions shorter than H timesteps.

D. Estimating Policy Calibration Parameters

We propose methods to empirically measure the unknown values in Equation 2 in simulation, without requiring an expert in the loop.

First, we estimate the unknown $\hat{\Sigma}_{k+1}$ in Equation 1 by sampling a segment of a recorded expert trajectory. We approximate $\hat{\Sigma}_{k+1}$ by sampling from the expert trajectories $\xi \sim p(\xi \mid \pi_{\theta^*})$ instead of from the noised expert trajectories $\xi \sim p(\xi \mid \pi_{\theta^*}, \Sigma_k^\alpha)$ because we find that it results in a $\hat{\Sigma}_{k+1}$ close to the true value in practice. This approximation lets us sample open loop, without a simulator. We set the policy's observations to the beginning of a trajectory segment then compare the policy's predicted action chunk with the expert's

future trajectory. From Equation 1, $\pi_\theta(s_t)$ is the policy's predicted action chunk and $\pi_{\theta^*}(s_t)$ is the expert trajectory.

We also estimate the robot's final closed-loop error α from Equation 2. One method to measure α in simulation is to roll out the policy initialized from the expert trajectory then measure the accumulated error over time. However, a suboptimal policy tends to quickly diverge from the expert without supervision. This may be from the multimodality of the task. Large deviations make measurements like the L2 norm ill-posed. We instead introduce a method to retain some of the policy while making it more likely for sampled trajectories to stay close to the expert intervention. To do this, we simulate the equilibrium of two competing forces: keeping the trajectory close to the expert trajectory and pulling the trajectory towards the policy's true error.

Let $x \in \mathbb{R}^d$ represent the state error relative to the expert trajectory. From Equation 7, the measured error between a sample of partial diffusion $\hat{a}_{t,\theta}$ and the expert data $a_{\theta^*,t}$ is bounded by the maximum policy predicted noise $\mathcal{K}$ scaled by $\sigma^2_{k_{sw}}$, where the switching step in partial diffusion $k_{sw} = \text{ToInteger}(\beta K)$. The noise schedule at the switching timestep of partial denoising for a diffusion model is $\sigma^2_{k_{sw}} = 1 - \bar{\alpha}_{k_{sw}} \in [0, 1]$. We use $\sigma^2_{k_{sw}}$ to model the interaction between the expert and the policy:

$$u_{\text{blend}} = \sigma^2_{k_{sw}} \cdot u_\theta + (1 - \sigma^2_{k_{sw}}) \cdot u_{\theta^*} \tag{8}$$

For an expert u_{θ^*} and policy u_θ, we assume the expert settles at 0 error and has an isotropic gain $K_{\theta^*} > 0$ for all dimensions d:

$$u_{\theta^*} = K_{\theta^*} \cdot (-x) \tag{9}$$

And we assume the policy settles towards an alternate goal W with isotropic gain $K_\theta > 0$:

$$u_\theta = K_\theta \cdot (W - x) \tag{10}$$

At the equilibrium, $u_{\text{blend}} = 0$ so

$$0 = \sigma^2_{k_{sw}} \cdot u_\theta + (1 - \sigma^2_{k_{sw}}) \cdot u_{\theta^*} \tag{11}$$

$$x = \frac{W \cdot \sigma^2_{k_{sw}}}{\sigma^2_{k_{sw}} + \frac{K_{\theta^*}}{K_\theta}(1 - \sigma^2_{k_{sw}})} \tag{12}$$

To recover the unknowns W and $\frac{K_{\theta^*}}{K_\theta}$, we sample the policy deviations after rolling out a "blended" trajectory for multiple blend ratios, where we compute partial denoising with respect to the time-matched future expert trajectory at each timestep, then execute the resulting blend action. We are interested in the settling state at equilibrium, so we discard the blended trajectories that diverge from the expert trajectory. Divergence is more likely at high blend ratios β. Between training iterations, we sample 13 blend ratios in [0.05, 0.90] once each and fit Equation 12. Although W is the settling error of the current policy, not of the final policy, we find that W tends to be stable across rounds k. See Figure 4.

An approximation of DART's α with the measured settling error W is:

$$\alpha = T||W||_2^2 \tag{13}$$

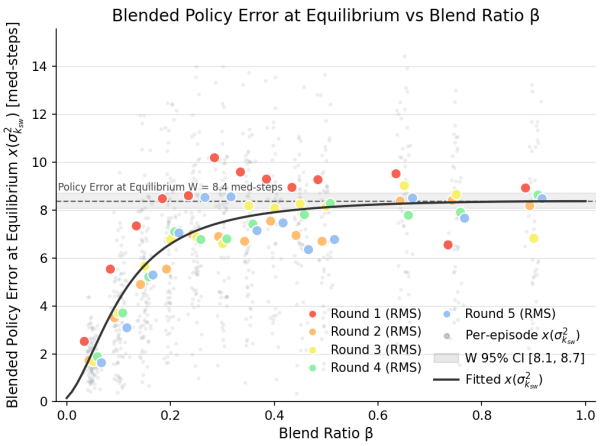


Fig. 4: Our fitted curve for estimating W on the planar reacher environment. As blend ratio $\beta \to 1$, noise variance fraction $\sigma^2_{k_{sw}} \to 1$ and thus our estimate $x(\sigma^2_{k_{sw}}) \to W$. We measure error in terms of median step size measured from the demonstration data.

And thus:

$$\hat{\Sigma}^\alpha_{k+1} = \frac{||W||_2^2}{\text{tr}\left(\hat{\Sigma}_{k+1}\right)} \hat{\Sigma}_{k+1} \tag{14}$$

Since we computed $\hat{\Sigma}_{k+1}$ at every timestep in the expert data, Σ^α_{k+1} is a per-step measurement of error. We compute a pooled noise measurement to reduce sample noise and also better match the per-round constant in DART [6]. For each of $m \in \mathcal{M}_{k+1}$ measurements of $\hat{\Sigma}_{k+1}$:

$$\bar{\Sigma}_{k+1} = \frac{1}{|\mathcal{M}_{k+1}|} \sum_{m \in \mathcal{M}_{k+1}} \hat{\Sigma}_m \tag{15}$$

$$\bar{\Sigma}^\alpha_{k+1} = \frac{||W||_2^2}{\text{tr}(\bar{\Sigma}_{k+1})} \bar{\Sigma}_{k+1} \tag{16}$$

Our policy calibration method is in Figure 2.

E. Generating Corrective Labels

We generate state-action pairs within the noise distribution that encourage the policy to return to the known data distribution. We assume access to a heuristic algorithm that connects a pair of nearby states. Action generation is done with respect to the demonstration dataset's velocity and acceleration statistics. For a perturbed starting state from our noise distribution, we label its action as an action chunk that starts from the perturbed state and returns to the original expert intervention trajectory. These action chunks can be generated cheaply because our policy's actions are joint positions. We ensure that these generated corrective action chunks are smooth and in distribution with the original dataset. We first associate the augmented state to a timestep of the associated expert intervention trajectory with a monotone closest-point projection. Then, we create a smooth trajectory between the augmented state and a state a few timesteps further in the associated expert trajectory state (computed with the median velocity in the dataset) by rolling out a critically-damped PD controller, with accelerations clamped to those in the original dataset's demonstrations.

Method	K=1	K=2	K=3	K=4	K=5
BC (no aggregation)	$68.0 \pm 0.2^{(1)}$				
HG-DAgger [7]	$79.9 \pm 0.5^{(5)}$	$81.9 \pm 0.7^{(5)}$	$83.3 \pm 1.5^{(5)}$	$85.3 \pm 0.7^{(5)}$	$87.3 \pm 1.0^{(5)}$
Fixed iso. noise (σ=2) [28]	$81.9 \pm 0.8^{(5)}$	$84.8 \pm 0.7^{(5)}$	$85.1 \pm 0.3^{(5)}$	$88.6 \pm 0.4^{(5)}$	$89.1 \pm 0.4^{(5)}$
Fixed iso. noise (σ=4)	$82.7 \pm 0.9^{(5)}$	$86.1 \pm 0.7^{(5)}$	$86.3 \pm 0.5^{(5)}$	$87.7 \pm 0.6^{(5)}$	$89.2 \pm 0.8^{(5)}$
Fixed iso. noise (σ=8)	$83.9 \pm 0.8^{(5)}$	$85.1 \pm 0.8^{(5)}$	$85.1 \pm 0.8^{(5)}$	$86.1 \pm 0.8^{(5)}$	$87.1 \pm 0.8^{(5)}$
Fixed iso. noise (σ=12)	$83.3 \pm 0.7^{(5)}$	$84.8 \pm 0.6^{(5)}$	$85.1 \pm 0.8^{(5)}$	$84.5 \pm 1.0^{(5)}$	$85.0 \pm 0.8^{(5)}$
Fixed iso. noise (σ=16)	$83.2 \pm 1.3^{(5)}$	$85.1 \pm 0.6^{(5)}$	$83.8 \pm 0.9^{(5)}$	$83.4 \pm 0.9^{(5)}$	$84.8 \pm 0.8^{(5)}$
Pooled $\bar{\Sigma}^{\alpha}$ (Ours)	$82.0 \pm 0.6^{(5)}$	$86.2 \pm 0.8^{(5)}$	$86.5 \pm 0.4^{(5)}$	$87.5 \pm 0.9^{(5)}$	$89.3 \pm 0.5^{(5)}$
Per-step $\hat{\Sigma}^{\alpha}$ (Ours)	$81.6 \pm 0.9^{(5)}$	$85.3 \pm 0.9^{(5)}$	$85.5 \pm 0.3^{(5)}$	$86.5 \pm 0.5^{(5)}$	$88.5 \pm 0.8^{(5)}$
Closed-loop pooled $\bar{\Sigma}^{\alpha}$ (Ours)	$81.6 \pm 1.0^{(1)}$	$83.3 \pm 0.2^{(1)}$	$85.9 \pm 0.5^{(1)}$	$88.8 \pm 0.2^{(1)}$	$89.3 \pm 0.7^{(1)}$

TABLE I: Mean±SE success rates on a 100-scenario planar reacher environment (3 evaluations per scenario). Most cells are averaged across 5 DAgger lineages$^{(5)}$. BC is trained on $\mathcal{D}_0$ (500 demonstrations). To prevent sequential optimization errors, all policy variants are fine-tuned from BC's 75k checkpoint for each round K (175k total steps). We sweep over isotropic noise levels and calibrated noise estimated before training for the round. Darker colors denote higher success. Our method matches the performance of the best noise level in the hyperparameter sweep without requiring extra expert data.

Method	K=1	K=2
BC (no aggregation)	$58.0 \pm 1.2^{(1)}$	
HG-DAgger [7]	$57.4 \pm 5.6^{(3)}$	$58.1 \pm 4.0^{(3)}$
Pooled $\bar{\Sigma}^{\alpha}$ (Ours)	$68.8 \pm 4.2^{(3)}$	$63.4 \pm 4.8^{(3)}$

TABLE II: Mean±SE success rates on a 100-scenario photo-realistic engine lever-reaching environment (3 evaluations per scenario). Following Table I's protocol, policy variants are fine-tuned from BC's 75k training step checkpoint to a total of 95k training steps. BC is trained on 500 demonstrations.

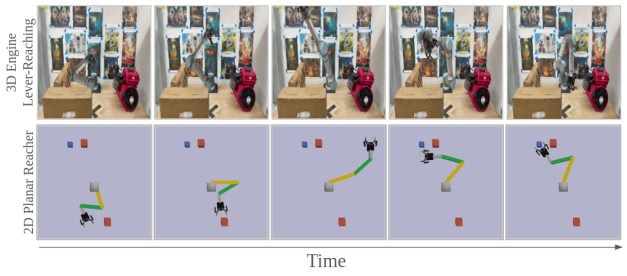


Fig. 5: We evaluate our method in two cluttered goal-reaching tasks with narrow passageways, where the robot often must untangle its links to make progress towards the goal. The 3D photorealistic physics simulation recreates our real mockup. We run ablations on the 3-link planar robot.

V. Experiments

We conduct a series of experiments to validate our method for data-efficient interactive imitation learning. Our primary research question is how our proposed augmentations to intervention data affect policy optimization given the same number of expert interventions. Our augmentations are based on a diffusion model's [43] denoising process. Specifically, we compare policy performance among a few data collection and learning methods: **(1)** Offline Behavior Cloning (BC), **(2)** Expert-gated data collection where the expert determines when to intervene and release control back to the policy (HG-DAgger [7]), **(3)** (2) but with a few fixed noise levels [28], and **(4)** (2) but noised with our predicted noise level at each round (Ours). Through these experiments, our aim is to demonstrate that data augmentations that match the distribution of errors that the policy will induce during evaluation time can improve interactive imitation learning without additional data collection.

We build our own simulator because existing benchmarks either lack an expert that can correct a policy from arbitrary states or lack the clutter needed to expose compounding error. We experiment on an engine lever-reaching task, where a UR5 manipulator arm navigates a cluttered, narrow passageway with randomized obstacles to reach an engine lever. All robot joints must move precisely to avoid collision, since an end effector-based action space would be under-constrained. We recreate a real engine lever-reaching task in a photorealistic simulation by following SplatSim [44]. We scan our physical experimental setup and objects with a smartphone to train Gaussian splats [45]. Then, we associate links in the robot's URDF with points in its splat. When a URDF moves in the PyBullet [46] physics simulation, we transform the associated points in its splat. In addition, we run ablations on a 2D planar reacher environment where a 3-link robot is tasked with reaching a goal end effector position while avoiding obstacles. Due to the clutter, the robot's embodiment often becomes the main collision obstacle in both tasks, and the robot must sometimes take a roundabout route to untangle its joints and links.

To improve reproducibility, we use a RRT-based oracle expert to generate intervention data in the loop when the evaluated policy is close to collision or stalled. We use this expert for both environments. The oracle plans a path from the current state to the goal, but we execute only a fraction of the plan before passing control back to the policy. This emulates how a human expert drives the robot just enough to return it to a safe state in human-gated DAgger. The oracle expert takes the optimal path under a deterministic criterion (for example, minimum end effector arc length). In addition, we smooth the RRT-generated trajectory with random shortcutting, trajectory optimization, corner smoothing, and speed, acceleration, and jerk limits enforced by TOPP-RA [47]. The path planner constrains the generated path to match the policy's induced pose and velocity at the handoff.

We conduct paired experiments where we first train a diffusion policy on the base dataset of 500 demonstrations on

randomized scenes, then iteratively collect intervention data and fine-tune the policy without using any noised data. Our results for the planar reacher task success rates across method configurations are shown in Table I. Our noised policies are fine-tuned from a checkpoint of the BC policy on the aggregate of: the base dataset $\mathcal{D}_0$, intervention data up to the given round of DAgger $\mathcal{D}_1 \dots \mathcal{D}_K$, and noise augmentations applied to the intervention data $\hat{\mathcal{D}}_1 \dots \hat{\mathcal{D}}_K$. This means that our noised policies are paired with the HG-DAgger results that do not use noise during training or data collection. In addition, the noised policies show performance gains over HG-DAgger and BC without having access to data from a noised policy. We present results where each policy is fine-tuned for the same number of steps within each environment, starting from the same checkpoint trained on the base dataset.

For the planar reacher task, BC without any dataset aggregation performs the worst at 68.0% success. The baseline data aggregation method HG-DAgger [7] increases the policy success rate from 79.9% to 87.3% success over five data collection iterations. In addition, most noised variants further increase the success rate over using $\mathcal{D}_k$ alone. Averaged over rounds and paired across lineages, pooled calibrated noise improves on HG-DAgger by 2.8 points ($t(4) = 5.75$, $p = 0.005$) and is equivalent within a ± 2 point margin to the best fixed noise level chosen in hindsight (TOST, $p \leq 0.01$).

In the 3D photorealistic engine lever-reaching environment, we evaluate our method on a policy with image observations. As an approximation, noised observations reuse the original image observation and only the joint position labels are noised. In Table II, BC obtains a 58.0% success rate. HG-DAgger performs similarly to BC despite additional intervention data: most failures are collisions happening within the first second of the episode, so intervention coverage is limited. In contrast, our proposed Pooled $\bar{\Sigma}^{\alpha}$ method scores higher than both BC and HG-DAgger when it has access to the same intervention data.

The optimal noise level tends to decrease as the policy gains competence through HG-DAgger rounds. In all five lineage reruns (collecting intervention data from scratch and seeding policy training), the best σ is 8–16 in $K = 1$ and is 2–4 by $K = 4$. Large noise levels like $\sigma = 12$ and 16 hurt the success rate in late rounds $K = 4$ and 5, while small noise levels like $\sigma = 2$ show an average 2 point increase in success in all DAgger rounds. Our method performs similarly to the best fixed isotropic noise ratio, but without sweeping over noise magnitudes. Our measured $\bar{\Sigma}^{\alpha}$ has a per-joint σ of 3.9–5.1 median per-step joint displacement in 2D planar reacher, close to the best-performing isotropic noise level $\sigma = 4$, and has a per-joint σ of 1.3–4.2 median steps in 3D engine lever-reaching in which the robot moves at similar velocities as the 2D planar reacher environment.

Another insight is that augmenting action labels may provide benefit in addition to only observation augmentations. Across all rounds k in the planar environment, even the smallest noise level in the sweep ($\sigma = 2$) increases success rate compared to HG-DAgger. This action relabeling was done in addition to the common practice of applying small Gaussian noise to the observations without changing action labels to simulate inaccurate signal readings.

VI. Conclusion

We present Policy-Calibrated DAgger, a method that applies policy-calibrated noise to data collected during iterative imitation learning, producing more robust policies without requiring additional expert supervision. Our method achieves higher success rates than behavior cloning and HG-DAgger, and matches performance with the best noise level in hindsight, without an expensive hyperparameter sweep over noise levels with an expert in the loop. Instead, we trade expert time for compute. We show that these trajectory data augmentations target policy errors during evaluation time and the increased success rates are consistent with reduced covariate shift. We use the policy as its own measure of noise, making our method agnostic to the semantics of individual action dimensions. Policy error is ill-defined when the trajectory to measure error with respect to does not exist in the policy's representation; however, we find that this gap is small in practice. Future work could derive partial denoising for other generative models such as flow matching models, apply our method to human-in-the-loop data collection, and extend our method to online noise estimation.

References

[1] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song, "Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots," *arXiv preprint arXiv:2402.10329*, 2024.

[2] H. Xiong, X. Xu, J. Wu, Y. Hou, J. Bohg, and S. Song, "Vision in action: Learning active perception from human demonstrations," in *Conference on Robot Learning*. PMLR, 2025, pp. 5450–5463.

[3] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. Ma, P. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J.-P. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. B. Simpson, Q. U. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Zhao, C. Agia, R. Baijal, M. G. Castro, D. L. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, M. Z. Irshad, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. M. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. Wang, Y. E. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Mart'in-Mart'in, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn, "Droid: A large-scale in-the-wild robot manipulation dataset," *ArXiv*, vol. abs/2403.12945, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:268531351

[4] T. Tao, M. K. Srirama, J. J. Liu, K. Shaw, and D. Pathak, "Dexwild: Dexterous human interactions for in-the-wild robot policies," *ArXiv*, vol. abs/2505.07813, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:278534598

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "Dart: Noise injection for robust imitation learning," in *Conference on robot learning*. PMLR, 2017, pp. 143–156.

[7] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.

[8] P. Wu, Y. Shentu, Q. Liao, D. Jin, M. Guo, K. Sreenath, X. Lin, and P. Abbeel, "Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation," *arXiv preprint arXiv:2503.07771*, 2025.

[9] S.-W. Lee, X. Kang, and Y.-L. Kuo, "Diff-dagger: Uncertainty estimation with diffusion policy for robotic manipulation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4845–4852.

[10] J. J. Liu, Y. Li, K. Shaw, T. Tao, R. Salakhutdinov, and D. Pathak, "Factr: Force-attending curriculum training for contact-rich policy learning," *arXiv preprint arXiv:2502.17432*, 2025.

[11] A. Bazhenov, S. Satsevich, S. Egorov, F. Khabibullin, and D. Tsetserukou, "Echo: An open-source, low-cost teleoperation system with force feedback for dataset collection in robot learning," *arXiv preprint arXiv:2504.07939*, 2025.

[12] T. Yoneda, L. Sun, G. Yang, B. Stadie, and M. Walter, "To the noise and back: Diffusion for shared autonomy," *arXiv preprint arXiv:2302.12244*, 2023.

[13] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert, "Learning monocular reactive uav control in cluttered natural environments," *2013 IEEE International Conference on Robotics and Automation*, pp. 1765–1772, 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:479635

[14] A. Yu, Z. Chen, P. Song, Z. Hong, H. Wang, D. Zhang, T. He, Y. Ding, and D. Zhang, "Wm-dagger: Enabling efficient data aggregation for imitation learning with world models," *arXiv preprint arXiv:2604.11351*, 2026.

[15] X. Zhang, M. Chang, P. Kumar, and S. Gupta, "Diffusion meets dagger: Supercharging eye-in-hand imitation learning," *arXiv preprint arXiv:2402.17768*, 2024.

[16] A. Haridas, K. Hamadeh, and S. C. B. Dash, "Dadagger: Disagreement-augmented dataset aggregation," *arXiv preprint arXiv:2301.01348*, 2023.

[17] Z. He, Y. Cao, and M. T. Ciocarlie, "Uncertainty comes for free: Human-in-the-loop policies with diffusion models," *ArXiv*, vol. abs/2503.01876, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:276774935

[18] R. Hoque, A. Balakrishna, E. R. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, "Thriftydagger: Budget-aware novelty and risk gating for interactive imitation learning," in *Conference on Robot Learning*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:237371142

[19] S.-W. Lee and Y.-L. Kuo, "Diff-dagger: Uncertainty estimation with diffusion policy for robotic manipulation," *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4845–4852, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:273502160

[20] S. Singi, Z. He, A. Pan, S. Patel, G. A. Sigurdsson, R. Piramuthu, S. Song, and M. T. Ciocarlie, "Decision making for human-in-the-loop robotic agents via uncertainty-aware reinforcement learning," *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 7939–7945, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257496009

[21] X. Xu, Y. Hou, Z. Liu, and S. Song, "Compliant residual dagger: Improving real-world contact-rich manipulation with human corrections," *Advances in Neural Information Processing Systems*, vol. 38, pp. 139 559–139 581, 2026.

[22] J. Li, W. Wang, Y. Peng, C. Shen, Y. Zhu, and Z. Xu, "Visual robotic manipulation with depth-aware pretraining," in *2024 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2024, pp. 843–850.

[23] K. Chen, E. Lim, K. Lin, Y. Chen, and H. Soh, "Don't start from scratch: Behavioral refinement via interpolant-based policy diffusion," *arXiv preprint arXiv:2402.16075*, 2024.

[24] Y. Tao, G. Qiao, D. Ding, and Z. Erickson, "Incremental learning for robot shared autonomy," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 20 797–20 804.

[25] A. Maddukuri, Z. Jiang, L. Y. Chen, S. Nasiriany, Y. Xie, Y. Fang, W. Huang, Z. Wang, Z. Xu, N. Chernyadev *et al.*, "Sim-and-real co-training: A simple recipe for vision-based robotic manipulation," *arXiv preprint arXiv:2503.24361*, 2025.

[26] X. Liu, D. Pathak, and K. M. Kitani, "Revolver: Continuous evolutionary models for robot-to-robot policy transfer," *arXiv preprint arXiv:2202.05244*, 2022.

[27] ——, "Herd: Continuous human-to-robot evolution for learning from human demonstration," *arXiv preprint arXiv:2212.04359*, 2022.

[28] L. Ke, J. Wang, T. Bhattacharjee, B. Boots, and S. Srinivasa, "Grasping with chopsticks: Combating covariate shift in model-free imitation learning for fine manipulation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6185–6191.

[29] A. Zhou, M. J. Kim, L. Wang, P. Florence, and C. Finn, "Nerf in the palm of your hand: Corrective augmentation for robotics via novel-view synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 907–17 917.

[30] L. Sun, J. Ji, X. Tan, and M. Walter, "Flashback: Consistency model-accelerated shared autonomy," in *Conference on Robot Learning*. PMLR, 2025, pp. 924–940.

[31] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, "Sdedit: Guided image synthesis and editing with stochastic differential equations," *arXiv preprint arXiv:2108.01073*, 2021.

[32] O. Shorinwa, J. Tucker, A. Smith, A. Swann, T. Chen, R. Firoozi, M. Kennedy III, and M. Schwager, "Splat-mover: Multi-stage, open-vocabulary robotic manipulation via editable gaussian splatting," *arXiv preprint arXiv:2405.04378*, 2024.

[33] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "Robogen: Towards unleashing infinite data for automated robot learning via generative simulation," *arXiv preprint arXiv:2311.01455*, 2023.

[34] J. Yang, J. J. Liu, Y. Li, Y. Khaky, K. Shaw, and D. Pathak, "Deep reactive policy: Learning reactive manipulator motion planning for dynamic environments," *arXiv preprint arXiv:2509.06953*, 2025.

[35] H. Oh, M. Murooka, T. Motoda, R. Nakajo, and Y. Domae, "Self-augmented robot trajectory: efficient imitation learning via safe self-augmentation with demonstrator-annotated precision," *Advanced Robotics*, pp. 1–15, 2026.

[36] C. Pan, L. Liang, D. Bauer, E. Cousineau, B. Burchfiel, S. Feng, and S. Song, "One demo is worth a thousand trajectories: Action-view augmentation for visuomotor policies," in *9th Annual Conference on Robot Learning*, 2025.

[37] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. J. Fan, and Y. Zhu, "Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 16 923–16 930.

[38] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," *arXiv preprint arXiv:2310.17596*, 2023.

[39] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu, "Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning," *arXiv preprint arXiv:2502.16932*, 2025.

[40] A. Jonnavittula, S. A. Mehta, and D. P. Losey, "Sari: Shared autonomy across repeated interaction," *ACM transactions on human-robot interaction*, vol. 13, no. 2, pp. 1–36, 2024.

[41] C. Zhou, J. Yang, D. J. Weber, and Z. Erickson, "Saps: Shared autonomy for policy steering by blending teleoperation with a pretrained vla," *arXiv preprint arXiv:2606.15568*, 2026.

[42] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[43] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[44] M. N. Qureshi, S. Garg, F. Yandun, D. Held, G. Kantor, and A. Silwal, "Splatsim: Zero-shot sim2real transfer of rgb manipulation policies using gaussian splatting," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 6502–6509.

[45] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis *et al.*, "3d gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.

[46] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016.

[47] H. Pham and Q.-C. Pham, "A new approach to time-optimal path parameterization based on reachability analysis," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645–659, 2018.